SYSTEM DESIGN

Chapter 57: Design a Distributed Cache

Personal Notes

Every large system uses caching. Ch 50 mentioned it as a standard building block; Ch 51's URL shortener and Ch 52's news feed leaned heavily on Redis for hot data. But we've been TREATING Redis as a magic box — a fast key-value store that just works. This chapter opens that box. We'll design one from scratch — Memcached / Redis at their essence — covering the internals that make distributed caches fast, consistent, and fault-tolerant.

The chapter starts small (a single-node LRU cache in a few dozen lines of Java) then scales it up: multiple nodes, consistent hashing, replication, persistence, client-side sharding. Along the way we'll finally do CONSISTENT HASHING properly — the algorithm that powers Amazon Dynamo, Cassandra, Memcached client libraries, and most distributed hash tables. Master this and you understand the internals of every distributed cache in production.

1. The Problem

Design an in-memory distributed key-value cache that stores hot data close to the application. Applications GET/SET/DELETE by key; the cache serves millions of ops/sec with sub-millisecond latency, scales horizontally to many nodes, tolerates node failures gracefully, and evicts old data automatically when memory is full.

```
Application:
   cache.set("user:42", userJson, ttl=3600)
   cache.get("user:42") → userJson (from RAM, ~1ms)

Behind the scenes:
   - Client hashes "user:42" to determine which cache node
   - Node stores in RAM with expiration timestamp
   - LRU eviction when memory fills
   - Replication to a peer for durability
   - Consistent hashing to minimize remapping when nodes change

Products in this category:
   Memcached — simple, blazing fast, no persistence
   Redis     — richer data types, optional persistence, pub-sub
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- GET, SET, DELETE by key
- SET with TTL (time-to-live in seconds)

- EVICT least-recently-used entries when memory is full
- MULTI-GET / batch operations for efficiency
- (Optional) Data-type extensions: lists, sets, hashes, sorted sets (Redis-style)
- (Optional) Publish/subscribe messaging (Redis-style)

2.2 Non-Functional Requirements

- Latency — p99 < 1 ms for GET, < 2 ms for SET
- Throughput — 100K+ ops/sec per node
- Availability — 99.99%+ (cache is on the hot path of every request)
- Consistency — EVENTUAL is usually fine (cache is a hint, not source of truth)
- Scale — total capacity ~1 TB across many nodes

2.3 Constraints to Confirm

- Persistence needed? — Redis yes, Memcached no. Trade-off: durability vs simplicity.
- Multi-tenant or single-purpose? — determines isolation, quota, security
- Max value size — typically 1 MB; larger values thrash memory + network
- Cross-region replication needed? — usually no; caches are regional

> **The critical clarification:** "Is the cache a hint, or the source of truth?" HINT (default) means the database has the canonical data; cache misses are recovered from DB. SOURCE OF TRUTH means data ONLY lives in the cache (e.g., session storage, real-time counters) — requires persistence + strong replication. This changes durability requirements dramatically.

3. Step 2 — Capacity Estimation

Throughput

```
    Assume the caller service handles 1M req/sec.
    Each request does ~5 cache operations → 5M cache ops/sec total.

    Per-node target: 100K ops/sec (Memcached does 200K+ in benchmarks)
    → Need ~50 cache nodes

    Latency budget: < 1 ms per GET (this is why we cache!)
```

Storage

```
    Total working set: 1 TB
    Per node: 20-40 GB RAM (fit comfortably in a mid-sized cloud VM)
    Per node capacity: ~50M entries at 500 B avg per entry

    Entry overhead: key (50 B) + value (variable) + metadata (24 B: ttl,
LRU pointers)
```

4. Step 3 — API Design

```
Basic operations:
  GET  key                          → value or NULL
  SET  key value [TTL=seconds]      → OK
  DEL  key                          → count deleted (0 or 1)
  EXISTS key                        → boolean

Batch:
  MGET key1 key2 key3 ...           → [value1, value2, value3]
  MSET k1 v1 k2 v2 ...              → OK

Atomic:
  INCR key                          → new count (creates if missing)
  DECR key
  SETNX key value                   → set only if key doesn't exist
(atomic)
  GETSET key value                  → set and return old value atomically

Meta:
  TTL key                           → seconds remaining, or -1 / -2
  EXPIRE key seconds                → set/update TTL
  STATS                             → hit rate, memory usage, evictions
```

Why SETNX exists: SET-IF-NOT-EXISTS is the ATOMIC primitive that makes distributed locks possible ("acquire the lock only if nobody else has it"). Every distributed system needs this. Redis's SETNX + TTL is how most people implement locks. Ch 51's URL shortener used PUT-IF-ABSENT for exactly this reason.

5. Building It — Step 1: Single-Node LRU Cache

Every distributed cache starts as a single-node cache. Get that right first, then scale it out. The core data structure: HASH MAP + DOUBLY-LINKED LIST. Together they give O(1) GET, SET, and eviction.

The Data Structure

```
    Hash Map:   key → Node (in the linked list)
    Linked List: MRU (most recently used)  ↔  ...  ↔  LRU

    GET("A"):
       1. Look up "A" in hash map → Node
       2. Move node to head (MRU)
       3. Return node.value
       Complexity: O(1)

    SET("A", v):
       1. If exists: update value, move to head
       2. Else: create node, add to head, put in map
```

```
        3. If size > capacity: remove tail node from list + map
        Complexity: O(1)
```

Java Implementation

```
public class LRUCache<K, V> {
    class Node {
        K key; V value; long expiresAt;
        Node prev, next;
    }

    private final Map<K, Node> map = new HashMap<>();
    private final Node head = new Node(), tail = new Node();
    private final int capacity;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public synchronized V get(K key) {
        Node n = map.get(key);
        if (n == null) return null;
        if (n.expiresAt > 0 && System.currentTimeMillis() > n.expiresAt) {
            remove(n); map.remove(key); return null;
        }
        moveToHead(n);
        return n.value;
    }

    public synchronized void set(K key, V value, long ttlMs) {
        Node n = map.get(key);
        if (n != null) {
            n.value = value;
            n.expiresAt = ttlMs > 0 ? System.currentTimeMillis() + ttlMs :
0;
            moveToHead(n);
            return;
        }
        n = new Node();
        n.key = key; n.value = value;
        n.expiresAt = ttlMs > 0 ? System.currentTimeMillis() + ttlMs : 0;
        map.put(key, n);
        addToHead(n);
        if (map.size() > capacity) {
            Node lru = tail.prev;
            remove(lru); map.remove(lru.key);
        }
    }
```

```
    private void moveToHead(Node n) { remove(n); addToHead(n); }
    private void addToHead(Node n)  {
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
    private void remove(Node n) {
        n.prev.next = n.next; n.next.prev = n.prev;
    }
}
```

The synchronized keyword is a rough hack: Real production caches use finer-grained locking (per-shard mutex) or lock-free data structures (Caffeine in Java uses ring buffers). A single mutex on every operation destroys throughput above ~50K ops/sec. But for the interview, calling out the concurrency concern is enough — you don't need to whiteboard the whole lock-free algorithm.

6. Scaling to Many Nodes — The Sharding Problem

One machine can hold ~40 GB of cached data. We need 1 TB total → ~25 nodes. How do we distribute keys across them? The naive answer: hash(key) % N. Let's see why that's a disaster.

The Naive Approach — hash(key) mod N

```
     4 nodes: N0, N1, N2, N3

     hash("user:42") = 100  → 100 % 4 = 0 → goes to N0
     hash("user:99") = 137  → 137 % 4 = 1 → goes to N1

     Fine — but what if we ADD a node? Now N = 5.

     hash("user:42") = 100  → 100 % 5 = 0 → still N0  ✓
     hash("user:99") = 137  → 137 % 5 = 2 → now N2   ✗ (moved!)

     In practice, ~80% of keys map to a NEW node after adding one.
     → Massive cache miss storm → DB gets hammered → thundering herd
```

Why cache miss storms are catastrophic: If 80% of keys suddenly miss the cache, ALL those requests fall through to your database. Your DB is designed for 20% of the load (assuming 80% cache hit); now it gets 5× that. It falls over. Cascading failure. This is why consistent hashing exists — to avoid this exact scenario.

7. Consistent Hashing — The Real Deep Dive

Consistent hashing was invented by Karger et al. in 1997 — specifically to solve the "how do we distribute cache keys?" problem. It's now the foundation of Amazon DynamoDB, Cassandra, memcached client libraries, Riak, and most distributed hash tables.

The Idea — A Hash Ring

```
Imagine a ring of hash values from 0 to 2^32 - 1.

Both KEYS and NODES hash to points on this ring.
Each key belongs to the NEXT NODE going clockwise.

              N0 (hash=100)
              ▲
              │
          ┌──────────────────┐
        /                    \
       │  ●key1 (hash=200)    │
       │    (belongs to N1)   │
        \                    /
          └──────────────────┘
              │
              ▼
              N1 (hash=300)

Add a new node N4 at hash=250:
   Only keys between 100 and 250 remap (from N1 to N4).
   Everything else stays. ~1/N of keys move — not 80%.
```

Basic Algorithm

```
class ConsistentHash {
    TreeMap<Long, Node> ring = new TreeMap<>();

    void addNode(Node n) {
        long h = hash(n.id);
        ring.put(h, n);
    }

    void removeNode(Node n) {
        ring.remove(hash(n.id));
    }

    Node getNodeForKey(String key) {
        if (ring.isEmpty()) return null;
        long h = hash(key);
        // Next entry >= h (clockwise); wrap to first if past the end
        Map.Entry<Long, Node> e = ring.ceilingEntry(h);
        if (e == null) e = ring.firstEntry();
        return e.getValue();
    }

    long hash(String s) {
        // Use MurmurHash3 or SipHash — NOT Java's default hashCode
        return MurmurHash.hash64(s);
```

```
    }
}
```

The Uneven Distribution Problem

```
With 4 nodes randomly placed on the ring, one node's arc might
cover 60% of the ring while another covers 5%.

Result: hot nodes overloaded, cold nodes idle.
Solution: VIRTUAL NODES.
```

8. Virtual Nodes — Making It Fair

Each physical node maps to MANY points on the ring (typically 100-200 "virtual nodes"). More points = more uniform distribution by law of large numbers. Each virtual node handles a tiny arc; the physical node's total load is the sum of all its arcs.

```
void addNode(Node n) {
    for (int i = 0; i < VIRTUAL_NODES_PER_PHYSICAL; i++) {
        long h = hash(n.id + "#" + i);
        ring.put(h, n);   // same physical node registered at 200 hash
positions
    }
}

// Now 4 physical nodes × 200 vnodes = 800 points on the ring.
// Standard deviation of load per physical node approaches 0
// as vnode count grows.
```

```
With 200 virtual nodes per physical:
   Standard deviation of load: ~5%
   Max load imbalance: ~15%
With just 1 virtual node per physical:
   Standard deviation: ~30%
   Some nodes get 3× the load of others.

200 vnodes is the industry standard sweet spot.
```

This is how Cassandra distributes tokens: Cassandra assigns each node 256 vnodes by default. Adding a new node means the new node's 256 vnodes randomly split the load — each existing node gives up roughly (1/N × 256) of its keys. Same math applies to DynamoDB partitions, memcached ketama hashing, and Redis Cluster hash slots (though Redis uses a fixed 16384 slots with explicit assignment).

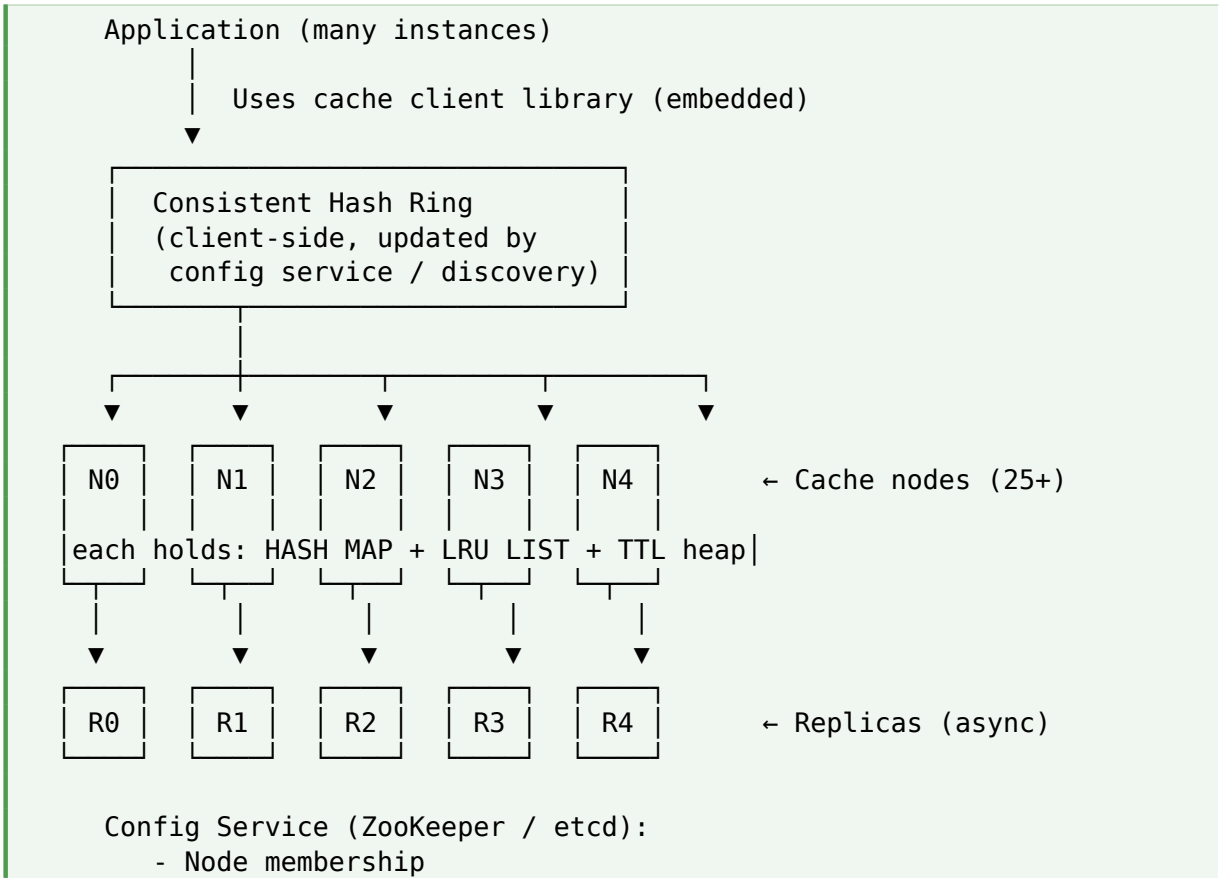
9. Client-Side vs Proxy-Based Sharding

Where does the sharding logic live? Two approaches, each used in production.

Aspect	Client-Side Sharding	Proxy-Based Sharding
Where sharding lives	Client library	Middle-tier proxy (Twemproxy, Envoy)
Latency	1 hop (client → node)	2 hops (client → proxy → node)
Config updates	Every client needs new config	Update proxy once
Language support	Need library per language	Language-agnostic
Failure isolation	Client sees direct errors	Proxy hides node failures
Used by	Memcached (ketama), Redis client libs	Twitter's Twemproxy, Envoy service mesh

Redis Cluster is different: Redis Cluster uses SMART CLIENT sharding with SERVER-SIDE redirection. Client makes its best guess; if wrong, server responds with MOVED/ASK redirect pointing to the correct node. Client updates its topology map. It's client-side sharding with server-side authority for topology changes.

10. Step 4 — High-Level Architecture



```
        - Ring topology
        - Health checks
```

Service Roles

- Cache nodes — hold data in RAM, LRU eviction, TTL expiry
- Config service — tracks live nodes, propagates topology changes
- Replicas — async copies of primary nodes for durability + failover
- Client library — knows the ring, picks the right node per key

11. Deep Dive: Eviction Algorithms

When memory fills up, which entry do you evict? Four common strategies, each optimal for different access patterns.

Policy	Strategy	Best for
LRU	Evict least recently used entry	Most workloads (default)
LFU	Evict least frequently used	Data with stable hot set
FIFO	Evict oldest by insertion order	Append-only workloads
Random	Evict a randomly-chosen entry	When LRU overhead isn't worth it
TTL only	Never evict; just expire by TTL	Session storage with defined lifespans
ARC / TinyLFU	Adaptive — combines LRU + LFU + frequency sketch	Netflix's Caffeine (Java) uses TinyLFU

> **LRU has a subtle failure mode:** If you scan a HUGE set of keys once (e.g., a batch job iterating all users), LRU evicts your entire hot set to make room for keys that will never be touched again. This is called "cache scan pollution." TinyLFU (which considers frequency) avoids this. Redis has LFU as an option specifically because of this problem.

12. Deep Dive: Replication for HA

A single node's failure loses ~40 GB of hot data — all your users start seeing DB latency. Replication guards against this. Common patterns:

Primary-Replica (Async)

- Every write goes to PRIMARY, then async to REPLICA
- Primary fails → promote replica; may lose the last few ms of writes
- Reads can go to either (usually stay on primary for consistency)

- Simple, low overhead — Redis default replication model

Multi-Master

- Any node accepts writes; propagates to peers
- Better availability, but conflict resolution required (last-write-wins, or CRDTs)
- Used by Cassandra, Riak — not typical for pure caches

Read Through Any Replica

For read-heavy caches, distribute reads across N replicas per shard → linear read scale. Writes still go to primary. Trade-off: replica lag can serve slightly stale data (usually fine for a cache).

```
    Sharded + Replicated architecture:

    Ring shards keys across 25 primary nodes.
    Each primary has 2 replicas.

    Total: 75 nodes, tolerating up to 2 failures per shard.
    Read capacity: 3× (all replicas serve reads).
    Write capacity: 1× (only primary accepts writes).
```

> **Replication ≠ persistence:** Async replication protects against SINGLE-NODE failure, but if all 3 replicas are in one datacenter and it goes offline, you lose everything. For cache-as-truth workloads (sessions), add DISK PERSISTENCE (Redis AOF or RDB) or a durable backing store (DynamoDB, Cassandra).

13. Deep Dive: Cache Patterns (from Ch 50, expanded)

Pattern	Mechanism	Trade-off
Cache-aside	App reads cache; on miss, reads DB and populates cache	Most common; default choice
Read-through	Cache library reads DB on miss automatically	Simpler app code; cache and DB must be tightly coupled
Write-through	Writes go to cache AND DB synchronously	Slower writes; consistent reads
Write-back	Writes to cache; async flushed to DB later	Very fast writes; data loss risk on crash
Refresh-ahead	Preemptively refresh popular entries before they expire	Predictable access patterns

14. Deep Dive: Persistence (Redis-Style)

Memcached: pure in-memory, no persistence — a node reboot loses everything. Redis: optional persistence via AOF or RDB. Understanding both helps you decide.

RDB — Snapshotting

- Fork a child process periodically; child writes snapshot to disk
- Very fast recovery (load snapshot on restart)
- But: can lose data between snapshots (5-15 min windows typical)

AOF — Append-Only File

- Every write appended to a log file (fsync policy: always / every-sec / never)
- Recovery: replay every command from the log
- Slower recovery but near-zero data loss
- File grows unbounded → periodic REWRITE compaction

Hybrid

Modern Redis defaults: use BOTH. RDB for fast recovery of the bulk; AOF for the recent tail. Load RDB first, replay AOF from that point. Best of both worlds.

15. Deep Dive: The Thundering Herd Problem

A super-popular key expires from cache. Simultaneously, 10,000 requests miss. All 10,000 try to fetch from DB. Your DB dies. You wanted the cache to protect the DB — instead it caused an attack on it.

Mitigations

- SINGLE-FLIGHT — only one request per key fetches from DB; others wait
- Probabilistic refresh — refresh the key slightly BEFORE TTL expires (say, 90% through TTL)
- Stale-while-revalidate — return the stale value briefly while refreshing async
- Randomize TTLs — add jitter so popular keys don't all expire together

```
// Single-flight pattern
Map<String, Future<V>> inflight = new ConcurrentHashMap<>();

V get(String key) {
    V v = cache.get(key);
    if (v != null) return v;

    Future<V> f = inflight.computeIfAbsent(key, k -> {
        return CompletableFuture.supplyAsync(() -> {
            V loaded = db.load(k);
            cache.set(k, loaded, TTL);
            return loaded;
```

```
        });
    });

    try {
        v = f.get();
    } finally {
        inflight.remove(key);
    }
    return v;
}
```

Related: the DOG-PILE effect: Similar to thundering herd but for RECURRING refresh operations. If your cache refreshes 1000 hot keys every 5 minutes, and they all refresh at the same second, you get a periodic DB spike. Stagger refresh times or add jitter.

16. Deep Dive: Cache Client Design

The client library is where most of the smarts live — consistent hashing, connection pooling, retry logic, failover. Getting the client wrong destroys the cache's benefits.

Key Client Responsibilities

- Maintain the consistent hash ring (updated from config service)
- Pool + reuse TCP connections to each node (don't open new connections per op)
- Batch: pipeline multiple operations to same node in one round-trip
- Timeout aggressively — if a node is slow, fail-fast to preserve latency SLA
- Circuit breaker — if a node is repeatedly failing, stop trying temporarily
- Failover — if primary is down, try replica

Pipelining

```
// Naive: 100 round-trips
for (String key : keys) {
    values.add(cache.get(key));
}

// Pipelined: 1 round-trip (per node)
Map<Node, List<String>> grouped = groupByNode(keys);
for (Node n : grouped.keySet()) {
    values.addAll(n.mget(grouped.get(n)));   // batch to that node
}
```

17. Additional Features

17.1 Pub/Sub

Redis-style: clients SUBSCRIBE to channels; PUBLISH sends a message to all subscribers. In-memory, at-most-once semantics. Great for cache invalidation broadcasts, presence notifications, lightweight event bus.

17.2 Transactions

Redis MULTI/EXEC groups commands; either all run or none. Not a real transaction (no rollback on error) — just a way to run commands atomically without interleaving with other clients.

17.3 Lua Scripts (referenced Ch 56)

Run arbitrary Lua code atomically on the server. Great for complex read-modify-write logic (token buckets, counters, custom eviction). One of Redis's most powerful features.

17.4 Data Types Beyond String

Redis extends beyond simple key-value: LISTS (queues), SETS (uniqueness), HASHES (objects), SORTED SETS (leaderboards), STREAMS (event logs), BITMAPS (counters), HYPERLOGLOG (approximate distinct counts). Each is optimized for specific access patterns.

18. Failure Modes

Failure	Mitigation
Cache node crash	Consistent hashing minimizes remapping. Client tries replica. Warm from DB.
Network partition	Client sees timeout; retries next node. Falls back to source of truth.
Cache-wide outage	Entire caching layer dies. DB gets full load — auto-scale + degrade gracefully.
Hot key (viral content)	Single node overwhelmed. Add replicas for that shard, or client-side caching.
Cache poisoning (bad data)	Wrong data cached. Manual purge + versioning key with a poison-safe id.
Ring reshuffling	Adding/removing nodes moves ~1/N keys. Warm slowly to avoid DB thundering herd.

19. Trade-offs Summary

Decision	Trade-off
Memcached vs Redis	Memcached = simpler, faster raw; Redis = rich types, persistence, pub/sub
Client-side vs proxy sharding	Client = 1 hop, per-language libs; proxy = language-agnostic, +1 hop latency
Async vs sync replication	Async = fast writes, may lose tail data; sync = safe, higher latency
LRU vs LFU eviction	LRU = simple, susceptible to scan pollution; LFU = handles scans, more state
Persistence yes/no	Persistence = restart safe, ~20% throughput hit; skip = pure speed
Virtual nodes count	More = better distribution, more memory + slower ring ops
Cache as hint vs source of truth	Hint = optional; source of truth = requires persistence + strong replication
Colocated vs remote cache	Local (in-process) = fastest; remote = shared across app instances

20. Common Mistakes

Mistake 1: Naive hash(key) % N sharding

Adding a node remaps 80% of keys → cache miss storm → DB cascading failure. Always use CONSISTENT HASHING or fixed-slot mapping (Redis Cluster style). Bring this up unprompted.

Mistake 2: Skipping virtual nodes

One physical node = one ring point → uneven distribution (some nodes 3× the load). Use 100-200 virtual nodes per physical for uniformity.

Mistake 3: No thundering herd protection

Hot key expires; thousands of misses; DB overloaded. Single-flight, probabilistic refresh, or jittered TTLs. Mention at least one strategy.

Mistake 4: Ignoring cache invalidation

Data changes in DB; cache still shows old value. When to invalidate? On write? On read (with version check)? TTL-only? Every design has different needs — address it explicitly.

Mistake 5: Storing huge values

A single 100 MB value in cache is a red flag. Blocks network I/O, causes eviction cascades, ruins hit rates. Keep values <1 MB. For bigger data, use blob store + cache a URL/pointer.

Mistake 6: One giant cache serving everyone

Different data has different access patterns (user data hot but small; product catalog cold but big). Separate cache clusters per use case can dramatically improve hit rates.

Mistake 7: Cache stampede on cold start

Restart your cache cluster → 100% miss rate → DB gets whole load until warm. Warm the cache slowly (throttle DB queries) or pre-populate from a persistent snapshot.

21. Best Practices

- Use CONSISTENT HASHING with 100-200 virtual nodes per physical for uniform distribution
- LRU eviction by default; consider LFU/TinyLFU if scan pollution is a concern
- Add SINGLE-FLIGHT or probabilistic refresh to prevent thundering herds on hot keys
- Randomize TTLs to avoid synchronized expiries
- Keep values under 1 MB — cache pointers to blob store for larger data
- Async replication for HA; add persistence only if cache is source of truth
- Client library batches operations via pipelining — huge throughput gain
- Circuit breakers on the client — fail-fast when a node is unreachable
- Separate cache clusters for different workloads — better isolation + hit rates
- Monitor hit rate, evictions/sec, p99 latency, memory pressure — these are your telltale metrics

22. Quick Reference

Concept	Meaning
LRU (Least Recently Used)	Evict entry not accessed for the longest time
LFU (Least Frequently Used)	Evict entry with lowest access frequency
TinyLFU	Frequency-aware admission policy (used by Caffeine)
Consistent hashing	Hash keys AND nodes to a ring; keys go to next node clockwise
Virtual nodes (vnodes)	One physical node → many ring points for uniformity
Hash ring	Circular hash space; foundation of DHTs
Client-side sharding	Client library routes to correct node — 1 hop
Proxy sharding	Middleware (Twemproxy, Envoy) routes — 2 hops but language-agnostic
Thundering herd	Many misses on same key at once → DB overload
Single-flight	One request loads key; others wait for the result

Concept	Meaning
Cache-aside	App reads cache first, DB on miss (most common)
Write-through	Sync write to cache AND DB — always consistent
RDB / AOF	Redis persistence — snapshot vs append-only log
Ketama	Consistent hashing scheme used by memcached clients

23. Related Interview Problems

Distributed cache patterns generalize to many other systems:

1. Design Memcached — same design, but no persistence, simpler protocol
2. Design Redis — add rich data types (lists, sorted sets), pub-sub, persistence
3. Design a Session Store — cache-as-source-of-truth; requires strong replication + persistence
4. Design a CDN — HTTP cache at edge; same eviction principles + geographic distribution
5. Design a Distributed KV Store (Dynamo) — cache-like but durable; consistent hashing + quorum
6. Design a Message Queue — related patterns (pub-sub, TTL, replication)
7. Design a Rate Limiter (Ch 56) — reuses in-memory KV state
8. Design a Feature Flag Cache — small hot data replicated everywhere

24. Final Takeaway

Distributed caches are a beautiful blend of algorithms + systems engineering. A single-node LRU is 50 lines of Java; making it distributed adds consistent hashing (with virtual nodes), replication for HA, thundering herd protection, careful client design. Get these five things right — LRU, consistent hashing with vnodes, replication, single-flight, and client-side pipelining — and you've built the internals of Memcached or Redis.

For interviews, walk it top-down: start with the single-node LRU (hash map + doubly-linked list, O(1) ops), scale to many nodes with consistent hashing + virtual nodes, address the thundering herd, discuss replication for HA, mention persistence trade-offs (Redis AOF/RDB). Then be ready for deep-dive follow-ups: "How does adding a node work? What if a node crashes mid-write? How do you invalidate globally?" These are the questions that distinguish memorized answers from real understanding.

> **Key Takeaway:** Distributed cache = HASH MAP + LRU LIST (single-node) + CONSISTENT HASHING with VIRTUAL NODES (sharding) + REPLICATION (HA) + SINGLE-FLIGHT (thundering herd). Client library handles ring lookup, connection pooling, pipelining. Add persistence (RDB/AOF) only if cache is source of truth. LRU is default; TinyLFU avoids scan pollution. Sharding: client-side (1 hop) or proxy (language-agnostic). Common patterns: cache-aside (default), write-through (consistent), write-back (fast). Never use naive hash(key) % N — 80% of keys remap on scaling.

25. What's Next?

Distributed cache done. Remaining designs and deep dives:

- Design a Notification System — cross-channel delivery, priorities, batching, dedup
- Design a Web Crawler — distributed fetching, politeness, dedup at scale
- Design a Search Autocomplete — trie-based (Ch 36), prefix caching, popularity
- Design a Payment System — idempotency, sagas, PCI compliance, eventual consistency
- Design a Metrics/Monitoring System (Datadog-style) — time-series data, retention tiers
- Design a Distributed Job Scheduler — Cron at scale, resilient task queues
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once
- Deep Dive: Consensus Algorithms — Raft (with worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing